If the `@ManyToOne` attribute is set to a valid `Post` entity reference:

```
Post post = entityManager.find(Post.class, 1L);
PostComment comment = new PostComment("My review");
comment.setPost(post);
entityManager.persist(comment);
```

Hibernate will generate an insert statement populating the `post_id` column with the identifier of the associated `Post` entity.

```
INSERT INTO post_comment (post_id, review, id) VALUES (1, 'My review', 2)
```

If the `Post` attribute is later set to null:

```
comment.setPost(null);
```

The `post_id` column will also be updated with a `NULL` value:

```
UPDATE post_comment SET post_id = NULL, review = 'My review' WHERE id = 2
```

Because the `@ManyToOne` association controls the foreign key directly, the automatically generated DML statements are very efficient.

Actually, the best-performing JPA associations always rely on the child-side to translate the JPA state to the foreign key column value.

This is one of the most important rules in JPA relationship mapping, and it will be further emphasized for `@OneToMany`, `@OneToOne` and even `@ManyToMany` associations.

10.3 @OneToMany

While the `@ManyToOne` association is the most natural mapping of the one-to-many table relationship, the `@OneToMany` association can also mirror this database relationship, but only when being used as a bidirectional mapping. A unidirectional `@OneToMany` association uses an additional junction table, which no longer fits the one-to-many table relationship semantics.

10.3.1 Bidirectional @OneToMany

The bidirectional `@OneToMany` association has a matching `@ManyToOne` child-side mapping that controls the underlying one-to-many table relationship. The parent-side is mapped as a collection of child entities.

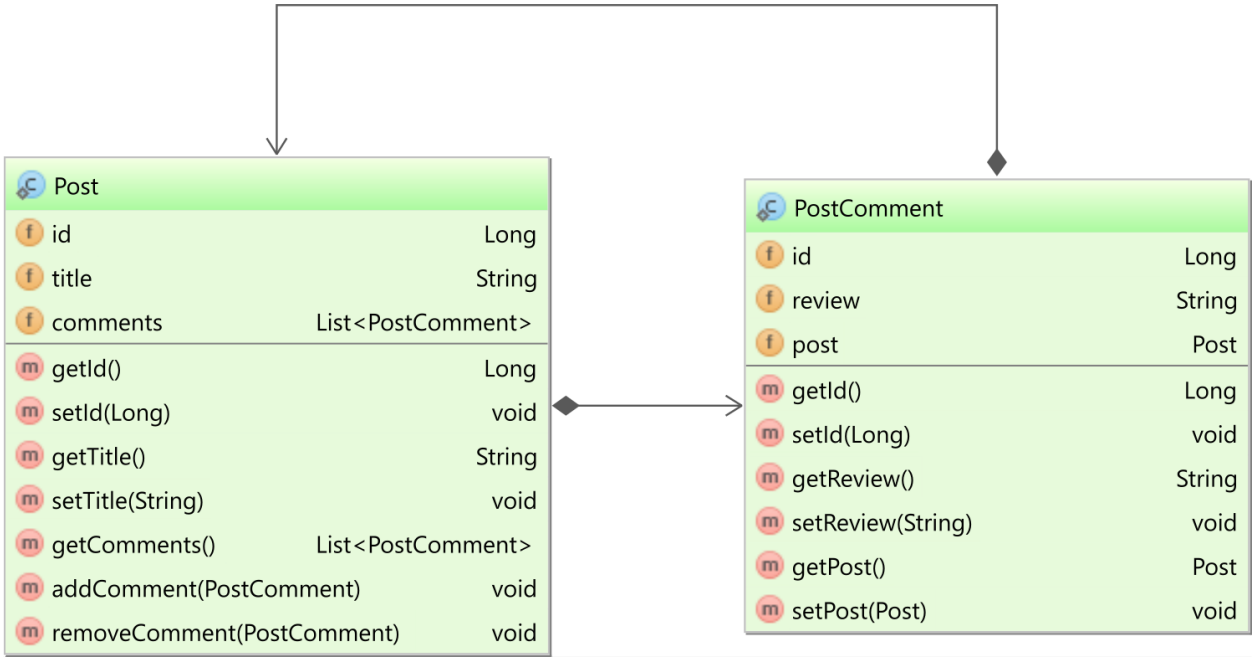

Figure 10.4: Bidirectional @OneToMany relationship

In a bidirectional association, only one side can control the underlying table relationship. For the bidirectional `@OneToMany` mapping, it is the child-side `@ManyToOne` association in charge of keeping the foreign key column value in sync with the in-memory Persistence Context. This is the reason why the bidirectional `@OneToMany` relationship must define the `mappedBy` attribute, indicating that it only mirrors the `@ManyToOne` child-side mapping.

```
@OneToMany(mappedBy = "post", cascade = CascadeType.ALL, orphanRemoval = true)
private List<PostComment> comments = new ArrayList<>();
```

Even if the child-side is in charge of synchronizing the entity state changes with the database foreign key column value, a bidirectional association must always have both the parent-side and the child-side in sync.

To synchronize both ends, it is practical to provide parent-side helper methods that add/remove child entities.

```
public void addComment(PostComment comment) {
    comments.add(comment);
    comment.setPost(this);
}

public void removeComment(PostComment comment) {
    comments.remove(comment);
    comment.setPost(null);
}
```

One of the major advantages of using a bidirectional association is that entity state transitions can be cascaded from the parent entity to its children. In the following example, when persisting the parent `Post` entity, all the `PostComment` child entities are persisted as well.

```
Post post = new Post("First post");

PostComment comment1 = new PostComment("My first review");
post.addComment(comment1);
PostComment comment2 = new PostComment("My second review");
post.addComment(comment2);

entityManager.persist(post);
```

```
INSERT INTO post (title, id) VALUES ('First post', 1)
INSERT INTO post_comment (post_id, review, id) VALUES (1, 'My first review', 2)
INSERT INTO post_comment (post_id, review, id) VALUES (1, 'My second review', 3)
```

When removing a comment from the parent-side collection:

```
post.removeComment(comment1);
```

The orphan removal attribute instructs Hibernate to generate a delete DML statement on the targeted child entity:

```
DELETE FROM post_comment WHERE id = 2
```

Equality-based entity removal

The helper method for the child entity removal relies on the underlying child object equality for matching the collection entry that needs to be removed.

If the application developer does not choose to override the default `equals` and `hashCode` methods, the `java.lang.Object` identity-based equality is going to be used. The problem with this approach is that the application developer must supply a child entity object reference that is contained in the current child collection.

Sometimes child entities are loaded in one web request and saved in a `HttpSession` or a Stateful Enterprise Java Bean. Once the Persistence Context, which loaded the child entity is closed, the entity becomes detached. If the child entity is sent for removal into a new web request, the child entity must be reattached or merged into the current Persistence Context. This way, if the parent entity is loaded along with its child entities, the removal operation will work properly since the removing child entity is already managed and contained in the children collection.

If the entity has not changed, reattaching this child entity will be redundant and so the `equals` and the `hashCode` methods must be overridden to express equality in terms of a unique business key. In case the child entity has a `@NaturalId` or a unique attribute set, the `equals` and the `hashCode` methods can be implemented on top of that. Assuming the `PostComment` entity has the following two columns whose combination render a unique business key, the equality contract can be implemented as follows:

```
private String createdBy;

@Temporal(TemporalType.TIMESTAMP)
private Date createdOn = new Date();

@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (o == null || getClass() != o.getClass()) return false;
    PostComment that = (PostComment) o;
    return Objects.equals(createdBy, that.createdBy) &&
           Objects.equals(createdOn, that.createdOn);
}

@Override
public int hashCode() {
    return Objects.hash(createdBy, createdOn);
}
```

Identifier-based equality

The java.lang.Object.equals[a] method Javadoc demands the strategy be *reflexive*, *symmetric*, *transitive*, and *consistent*.

While the first three equality properties (*reflexive*, *symmetric*, *transitive*) are easier to achieve, especially with the `java.util.Objects`[b] `equals` and `hashCode` utilities, *consistency* requires more diligence.

For a JPA or Hibernate entity, *consistency* means that the equality result is *reflexive*, *symmetric* and *transitive* across all entity state transitions (e.g. new/transient, managed, detached, removed).

If the entity has a `@NaturalId` attribute, then ensuring *consistency* is simple since the natural key is assigned even from the transient state, and this attribute never changes afterward. However, not all entities have a natural key to use for equality checks, so another table column must be used instead.

Luckily, most database tables have a primary key, which uniquely identifies each row of a particular table. The only caveat is to ensure *consistency* across all entity state transitions.

A naive implementation would look like this:

```
@Entity
public class Post {

    @Id @GeneratedValue
    private Long id;

    //Getters and setters omitted for brevity

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Post)) return false;
        return Objects.equals(id, ((Post) o).getId());
    }

    @Override
    public int hashCode() {
        return Objects.hash(id);
    }
}
```

[a] https://docs.oracle.com/javase/8/docs/api/java/lang/Object.html#equals-java.lang.Object-
[b] https://docs.oracle.com/javase/7/docs/api/java/util/Objects.html